

TP 4.b – Morning Briefing : pré-requis et configuration des comptes externes

Objectif de ce document : vous accompagner, **avant la session pratique sur n8n**, dans la création de tous les comptes et clés API nécessaires au workflow *Morning Briefing*.

Sommaire

- [1. Que va-t-on construire ?](#)
 - [2. Architecture cible](#)
 - [3. Vue d'ensemble des comptes à créer](#)
 - [4. Créer une clé API Groq \(LLM\)](#)
 - [5. Créer un compte Gmail dédié \(sandbox\)](#)
 - [6. Configurer Google Cloud Console](#)
 - [6.1 Créer un projet GCP](#)
 - [6.2 Activer les 3 APIs nécessaires](#)
 - [6.3 Configurer l'écran de consentement OAuth](#)
 - [6.4 Créer le client OAuth Web pour n8n](#)
 - [7. Configurer Slack](#)
 - [7.1 Créer un workspace Slack dédié](#)
 - [7.2 Créer l'application Slack](#)
 - [7.3 Activer Incoming Webhooks](#)
 - [7.4 Tester le webhook depuis le terminal](#)
 - [8. Checklist finale avant la séance](#)
 - [9. Pour aller plus loin](#)
-

1. Que va-t-on construire ?

Le **Morning Briefing** est un agent IA qui s'exécute automatiquement chaque matin (8h30, du lundi au vendredi) et qui produit pour vous un **résumé exécutable de votre journée**, posté dans Slack.

Concrètement, l'agent va :

1. **Lire votre agenda Google Calendar** – récupérer les 3 prochaines réunions du jour.
2. **Enrichir chaque réunion** – chercher dans Gmail les derniers échanges avec les participants, et dans Drive les documents liés au sujet.
3. **Scanner votre boîte Gmail** – détecter les mails non lus depuis 24 h.

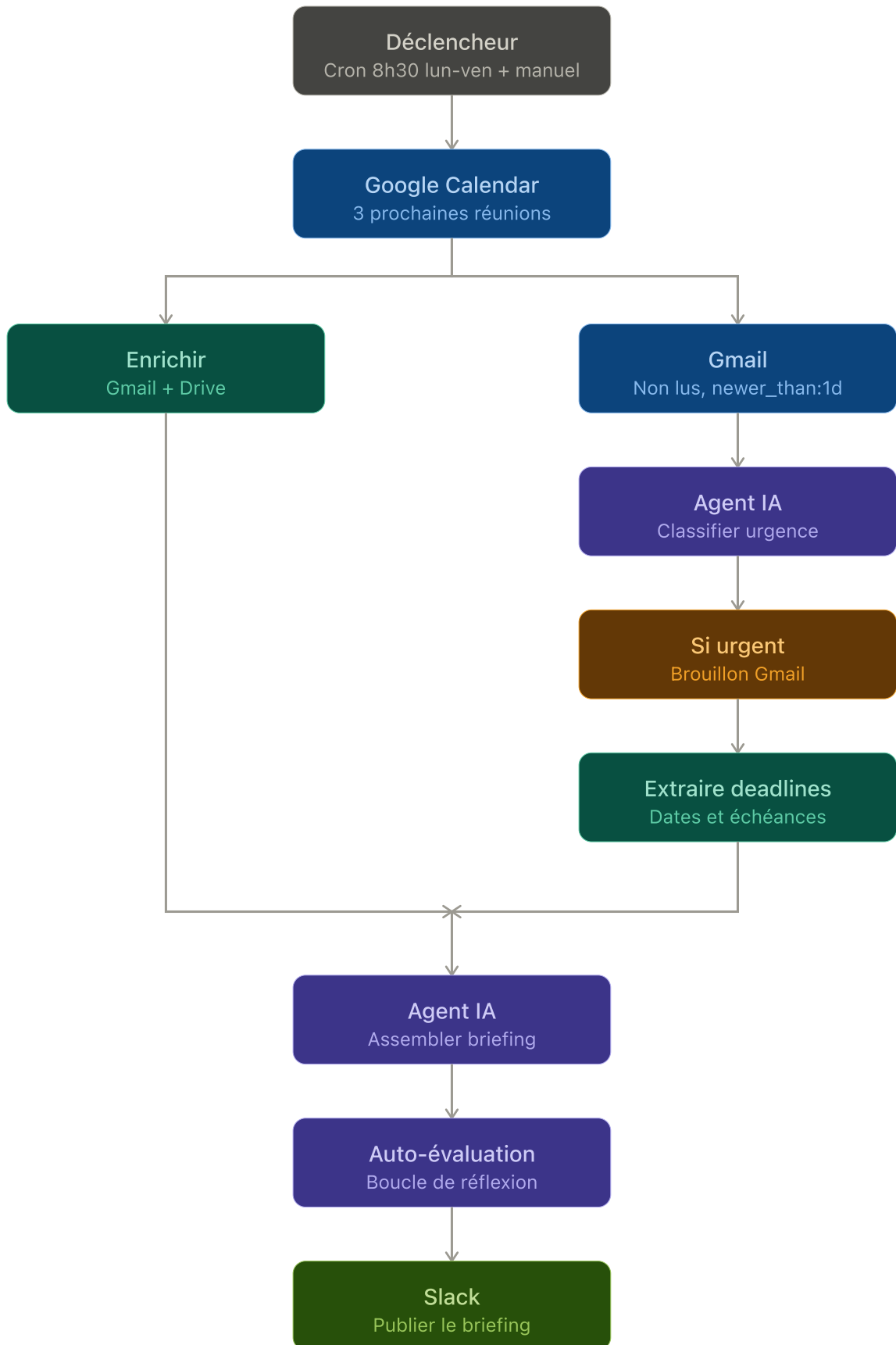
4. **Classifier l'urgence** de chaque mail via un LLM (Groq).
5. **Préparer des brouillons de réponse** (drafts Gmail, jamais envoyés automatiquement) pour les mails urgents.
6. **Extraire les deadlines** mentionnées dans les mails récents.
7. **Assembler un briefing markdown** clair et concis.
8. **Auto-évaluer** la qualité du briefing (boucle de réflexion).
9. **Poster le briefing dans Slack** sur le canal de votre choix.

Pourquoi ce use case en formation ?

Le briefing matinal est un **workflow récurrent, lourd en intégrations**. C'est exactement le terrain où n8n brille : connecteurs Gmail/Calendar/Drive/Slack natifs, OAuth géré via UI, **Schedule Trigger** intégré, retries et reprise sur erreur prêts à l'emploi.

Là où le notebook Python du TP 4.a vous apprend la **mécanique interne** d'un agent (boucle ReAct, gestion de tools, mémoire), ce TP vous apprend les **réflexes opérationnels** : OAuth en 3 clics, cron natif, idempotence et observabilité fournies par le runtime.

2. Architecture cible



L'agent utilisera **5 services externes** : Google Calendar, Gmail, Google Drive, Groq (LLM), Slack. Avant de pouvoir construire le workflow dans n8n, il faut donc **configurer ces 5 services** – c'est l'objet du présent document.

3. Vue d'ensemble des comptes à créer

Voici la carte mentale de ce que vous allez faire dans les sections suivantes :

#	Service	Pourquoi	Coût	Temps
1	Compte Groq + clé API	Le LLM qui pilotera l'agent (modèle <code>llama-3.3-70b</code>)	Gratuit	3 min
2	Nouveau compte Gmail dédié	Compte sandbox pour les tests – JAMAIS votre compte perso/pro	Gratuit	3 min
3	Projet Google Cloud sur le compte sandbox	Conteneur des APIs et de l'OAuth	Gratuit	2 min
4	3 APIs Google activées (Gmail, Calendar, Drive)	Permettre à n8n d'appeler ces services	Gratuit	3 min
5	Écran de consentement OAuth + client OAuth Web	Permettre à n8n de demander l'autorisation à l'utilisateur	Gratuit	7 min
6	Workspace Slack dédié	Espace de réception du briefing – pas votre Slack pro	Gratuit	3 min
7	App Slack + Incoming Webhook	URL de POST sur laquelle n8n enverra le briefing	Gratuit	5 min

Note pédagogique : pourquoi tant de comptes "sandbox" ?

Pendant le TP, votre agent va **lire vos mails, créer des brouillons, chercher dans votre Drive et poster dans Slack**. Si vous utilisez vos comptes professionnels, le moindre bug de classification ou de prompt mal calibré peut générer un brouillon embarrassant, polluer votre vraie boîte, ou spammer un canal Slack pro.

Créer des comptes jetables prend 10 min et vous évite des incidents. C'est un **réflexe métier** à intégrer dès maintenant : on ne teste jamais un agent IA sur des données de

4. Créer une clé API Groq (LLM)

Groq est un fournisseur de LLM qui propose un **tier gratuit générique** (plusieurs milliers de requêtes par jour) avec des modèles open-source rapides comme Llama 3.3 70B. C'est idéal pour la formation : pas de carte bancaire requise, latence faible.

Procédure

1. Allez sur <https://console.groq.com>.
2. Cliquez **Sign up** (en haut à droite).
3. Créez un compte avec votre email habituel ou via Google/GitHub.
4. Vous arrivez sur le dashboard Groq.
5. Dans le menu de gauche, cliquez **API Keys**.
6. Cliquez **Create API Key** (bouton vert).
7. Donnez-lui un nom : `tp4-briefing-agent`.
8. **Copiez immédiatement la clé** (format : `gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX`).

⚠ **Cette clé n'est affichée qu'une seule fois.** Si vous la perdez, vous devrez la régénérer.

Stockage temporaire de la clé

Créez un fichier de notes local (par exemple `apiKey.txt` ou un Notes Mac) où vous collerez **toutes les clés** au fur et à mesure :

```
GROQ_API_KEY = gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
GMAIL_SANDBOX_EMAIL = ...
GCP_OAUTH_CLIENT_ID = ...
GCP_OAUTH_CLIENT_SECRET = ...
SLACK_WEBHOOK_URL = ...
```

Note pédagogique : Groq vs OpenAI

Groq expose une **API compatible OpenAI** : même endpoints, même schéma de requêtes. Concrètement, ça veut dire qu'on peut switcher de fournisseur en changeant juste deux variables : la `Base URL` et la clé API. C'est devenu le **standard de facto** des nouveaux fournisseurs LLM (Together, Mistral, Anyscale...).

Pour la formation, on utilise Groq parce qu'il est **gratuit, rapide et sans carte bancaire**. En production, le choix dépendra du modèle visé, des contraintes de latence et des SLAs.

5. Créer un compte Gmail dédié (sandbox)

⚠ **Étape critique.** N'utilisez PAS votre compte Gmail personnel ni professionnel.

Procédure

1. Ouvrez une **fenêtre de navigation privée** (pour éviter les conflits avec votre compte Gmail principal).
2. Allez sur <https://accounts.google.com/signup>.
3. Remplissez le formulaire :
 - **First name** : (ou ce que vous voulez)
 - **Last name** : (ou ce que vous voulez)
 - **Username** : choisissez un identifiant disponible, par exemple
 - **Password** : un mot de passe robuste (notez-le)
4. Continuez les étapes (numéro de téléphone, date de naissance – Google peut accepter un compte sans téléphone, ou en demander un selon votre IP).
5. Une fois le compte créé, **restez connecté à ce compte** dans la fenêtre privée. Toutes les manipulations Google Cloud qui suivent se feront avec **CE compte**.

Note pédagogique : pourquoi un compte 100 % séparé ?

Au-delà de la sécurité, un compte dédié vous permet :

- D'avoir un **environnement vide et prévisible** (vous savez exactement ce qu'il y a dans la boîte).
- De pouvoir **supprimer le compte à la fin du TP** sans regret (Google permet de supprimer un compte → toutes les données + tokens OAuth + projets GCP disparaissent).
- De **partager les écrans** pendant la formation sans révéler vos vrais mails.

En entreprise, le pendant de cette pratique s'appelle un **compte de service** ou un **compte technique** : un compte non-nominatif dédié à une intégration. C'est ce que vous mettrez en place en prod.

6. Configurer Google Cloud Console

Cette section est la plus longue (~ 12 min). Elle se déroule en **4 sous-étapes** : projet, APIs, écran de consentement, client OAuth.

Avant de commencer, vérifiez que vous êtes bien connecté sur Google avec **votre compte sandbox** (et non votre compte habituel). Le mieux est de le faire dans la même fenêtre privée que celle utilisée pour créer le compte.

Allez sur <https://console.cloud.google.com>. Acceptez les conditions d'utilisation si demandé.

6.1 Créer un projet GCP

1. En haut à gauche, à côté du logo *Google Cloud*, cliquez sur le **sélecteur de projet** (texte du genre « *Sélectionnez un projet* »).
2. Dans la modale qui s'ouvre, cliquez **NEW PROJECT** (en haut à droite).
3. Remplissez :
 - **Project name** : `tp4-briefing-agent`
 - **Location** : laissez `No organization`
4. **CREATE**.
5. Attendez 10-30 secondes la notification de création (en haut à droite).
6. **Resélectionnez explicitement le projet** depuis le sélecteur en haut – Google ne switch pas toujours automatiquement.

✅ Le sélecteur en haut doit afficher `tp4-briefing-agent`.

6.2 Activer les 3 APIs nécessaires

Dans la barre de recherche en haut, tapez :

API **Library**

→ cliquez le résultat « **Bibliothèque d'API** » (ou *API Library*).

Activez les 3 APIs suivantes, **une par une** :

#	API à rechercher	Action
1	<code>Gmail API</code>	Cliquer le résultat → ACTIVER
2	<code>Google Calendar API</code>	Cliquer le résultat → ACTIVER
3	<code>Google Drive API</code>	Cliquer le résultat → ACTIVER

Pour chacune : recherche → résultat → bouton **ACTIVER** (bleu) → revenir à la Library (flèche ← du navigateur).

✅ Vérifiez : ☰ → **APIs et services** → **APIs et services activés** → les 3 APIs apparaissent dans la liste.

6.3 Configurer l'écran de consentement OAuth

L'écran de consentement, c'est ce que verra l'utilisateur quand n8n lui demandera l'autorisation d'accéder à ses données Google. Sans cette configuration, on ne peut pas créer de client OAuth.

Dans la barre de recherche, tapez :

OAuth consent screen

→ cliquez le premier résultat. Vous arrivez sur la **Google Auth Platform**.

Cliquez **Premiers pas** (bouton bleu, en bas).

Vous entrez dans un assistant en 4 étapes :

Étape 1 – App Information

Champ	Valeur
App name	tp4-briefing-agent
User support email	sélectionnez votre email sandbox dans le dropdown

→ **NEXT**.

Étape 2 – Audience

- **External** ← sélectionnez celle-ci (Internal est grisé pour les comptes Gmail perso).

→ **NEXT**.

Étape 3 – Contact information

Champ	Valeur
Email addresses	retapez votre email sandbox

→ **NEXT**.

Étape 4 – Finish

- Cochez « **I agree to the Google API Services User Data Policy** ».
- → **CONTINUE** → **CREATE**.

Ajouter l'utilisateur test (CRITIQUE)

Vous arrivez sur le dashboard avec un menu de gauche.

1. Menu de gauche → cliquez **Audience**.
2. Scrollez jusqu'à la section **Test users**.
3. Cliquez **+ ADD USERS**.
4. Saisissez votre email sandbox.
5. **SAVE**.

✅ La page **Audience** doit afficher :

- **Type d'utilisateur :** `Externe`
- **Test users :** 1 utilisateur (votre email sandbox)
- **État implicite :** `Testing` (la phrase « *Quand l'état de publication est défini sur "Test"...* » le confirme)

⚠ **Ne cliquez PAS sur "Publier l'application".** Publier déclenche une validation Google de plusieurs jours et n'est pas nécessaire pour notre usage.

Note pédagogique : Testing vs Production

- **Testing :** 1 à 100 utilisateurs déclarés en *Test users*, aucune validation Google requise. Parfait pour usage interne ou pilote.
- **Production :** nombre illimité d'utilisateurs, mais **validation Google obligatoire** pour les *sensitive scopes* comme `gmail.modify` (5-15 jours, demande un domaine vérifié, une politique de confidentialité publique, et une vidéo de démonstration).

Pour 95 % des cas d'usage interne d'entreprise, on reste en **Testing à vie**. C'est gratuit, ça fonctionne, et ça évite l'audit Google.

6.4 Créer le client OAuth Web pour n8n

C'est l'étape qui produit les deux secrets que n8n utilisera : **Client ID** et **Client Secret**.

1. Dans le menu de gauche de la Google Auth Platform, cliquez **Clients**.
2. Cliquez **+ CREATE CLIENT** (en haut).
3. Remplissez :

Champ	Valeur
Application type	<code>Web application</code> ⚠ PAS Desktop
Name	<code>n8n-briefing-agent</code>

4. Scrollez jusqu'à **Authorized redirect URIs** → **+ ADD URI**.
5. Collez **exactement** :

```
http://localhost:5678/rest/oauth2-credential/callback
```

⚠ Vérifications :

- `http://` (pas `https://`)
- `localhost` (pas `127.0.0.1`, pas votre IP)
- Port `5678`
- Aucun espace, aucun `/` final supplémentaire

6. CREATE.

7. Une modale apparaît avec :

- **Your Client ID** : `xxxxxxxxxxxx-xxxx.apps.googleusercontent.com`
- **Your Client Secret** : `GOCSPX-xxxxxxxxxxxxxxxx`

8. **Copiez les deux valeurs immédiatement** dans votre fichier de notes.

Note pédagogique : Web vs Desktop

Le notebook Python du TP 4.a utilisait un client OAuth de type **Desktop**, qui ouvre un mini-serveur local éphémère pour récupérer le code d'autorisation. n8n, lui, est lui-même un serveur HTTP qui tourne en permanence – il a donc besoin d'un client de type **Web application** avec une URL de redirection stable.

C'est la **différence majeure** entre les deux setups OAuth Google. Si vous obtenez l'erreur `redirect_uri_mismatch` côté n8n, c'est presque toujours que vous avez utilisé un client Desktop, ou que l'URI de redirection ne matche pas au caractère près.

7. Configurer Slack

Slack est beaucoup plus simple à configurer que Google : pas d'OAuth complet, juste une URL de webhook nominative.

7.1 Créer un workspace Slack dédié

⚠ **N'utilisez PAS votre Slack professionnel.** Créez un workspace personnel dédié au TP.

1. Allez sur <https://slack.com/get-started>.
2. Saisissez votre email (vous pouvez utiliser votre email sandbox Gmail, ou un autre).
3. Confirmez avec le code à 6 chiffres reçu par email.
4. Quand on vous demande « **Nommer votre espace de travail** », mettez : `tp4-formation` (ou `prenom-tp4`).
5. **Votre nom** : libre.
6. **Inviter d'autres personnes** → **Skip** (lien discret en bas).
7. **Créer un canal** : nommez-le `briefings-test` (ce sera le canal cible du webhook).
8. Vous arrivez dans Slack.

Note pédagogique : workspace ≠ app

Le **workspace** est l'environnement Slack entier, équivalent d'une "organisation". L'**app** est une intégration installée à l'intérieur. Une même app peut être installée dans plusieurs workspaces – c'est comme ça que des outils du marketplace Slack (Notion, Asana, Google Calendar) sont distribués.

7.2 Créer l'application Slack

L'app **ne se crée pas dans Slack lui-même**, mais sur le portail développeur Slack.

1. Ouvrez un **nouvel onglet** : <https://api.slack.com/apps>.
2. Connectez-vous avec le **même compte Slack** que celui qui possède le workspace `tp4-formation`.
3. Cliquez **Create New App** (vert, en haut à droite).
4. Choisissez **From scratch**.
5. Remplissez :

Champ	Valeur
App Name	<code>tp4-briefing-agent</code>
Pick a workspace	<code>tp4-formation</code> (votre workspace)

6. **Create App**.

Vous arrivez sur la page **Basic Information** de votre nouvelle app. **Ignorez les App Credentials affichés** (App ID, Client Secret, Signing Secret) – on n'en a pas besoin pour les Incoming Webhooks.

7.3 Activer Incoming Webhooks

1. Dans le menu de gauche, sous **Features**, cliquez **Incoming Webhooks**.
2. En haut à droite, basculez le toggle **Activate Incoming Webhooks** sur **On**.
3. Scrollez en bas → cliquez **Add New Webhook to Workspace**.
4. Une page Slack s'ouvre :
 - Sélectionnez le canal `#briefings-test`.
 - Cliquez **Allow**.
5. Vous revenez sur la page Incoming Webhooks. Une nouvelle ligne apparaît dans **Webhook URLs for Your Workspace** avec :
 - Le canal cible
 - L'URL `https://hooks.slack.com/services/T.../B.../xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx`
 - Un bouton **Copy**
6. **Copiez l'URL** dans votre fichier de notes.

Note pédagogique : Incoming Webhook vs OAuth complet

Slack offre deux mécanismes d'intégration :

- **Incoming Webhooks** (notre choix) : URL nominative, attachée à un canal précis. Idéal pour POSTER. 3 min de setup, aucun scope.
- **Slack OAuth + Bot Token** : pour un bot qui doit **lire** les messages, **interagir** (boutons, modals, slash commands), ou poster dynamiquement dans plusieurs canaux. Plus puissant, beaucoup plus complexe.

Pour un briefing quotidien qui fait juste POSTER, le webhook suffit largement. **Principe à retenir : on utilise toujours le mécanisme le moins puissant qui résout le problème.**

7.4 Tester le webhook depuis le terminal

Avant même de toucher à n8n, validons que le webhook fonctionne. Dans votre terminal :

```
curl -X POST -H 'Content-type: application/json' --data '{"text":"Hello depuis le terminal - test webhook"}' "COLLE_TON_URL_ICI"
```

(remplacez `COLLE_TON_URL_ICI` par votre URL webhook complète, **gardez les guillemets**)

→ Réponse attendue : `ok` (en texte brut, pas de JSON)

→ Côté Slack : le message arrive dans `#briefings-test`.

Note pédagogique : pièges shell classiques

Si vous obtenez `curl: (3) URL rejected: Malformed input to a URL function`, c'est presque toujours :

1. Une **continuation de ligne** avec `\` cassée par un espace invisible → mettez la commande **sur une seule ligne**.
2. Des **guillemets manquants** autour de l'URL → l'URL contient des `&`, `?`, `=` qui peuvent être interprétés par le shell.

Réflexe : en démo ou en debug, **toujours** taper les `curl` sur une seule ligne, **toujours** entourer les URLs de guillemets.

8. Checklist finale avant la séance

Avant de venir à la séance pratique sur n8n, vérifiez que vous avez bien dans votre fichier de notes (ou équivalent) :

- ☐ **Clé API Groq** : format `gsk_...`
- ☐ **Email Gmail sandbox** : un nouvel email dédié, jamais utilisé ailleurs
- ☐ **Mot de passe** du compte Gmail sandbox
- ☐ **Projet GCP** `tp4-briefing-agent` créé et **3 APIs activées** (Gmail, Calendar, Drive)
- ☐ **Écran de consentement OAuth** configuré, en mode `Testing`, avec votre email sandbox dans **Test users**

- ☐ **Client OAuth Web** créé : `Client ID` (`...apps.googleusercontent.com`) et `Client Secret` (`GOCSPX-...`) copiés
 - ☐ **Workspace Slack** `tp4-formation` créé avec un canal `#briefings-test`
 - ☐ **App Slack** `tp4-briefing-agent` créée et **Incoming Webhook** activé
 - ☐ **URL webhook Slack** copiée et **testée** au `curl` (réponse `ok` reçue)
- ✅ Si toutes les cases sont cochées, vous êtes prêt(e) pour la séance pratique. Nous attaquerons directement la **construction du workflow dans n8n** sans perdre de temps sur les comptes externes.

9. Pour aller plus loin

Pourquoi ces choix d'outils ?

Choix	Alternative envisageable	Pourquoi celui-ci pour le TP
Groq	OpenAI, Anthropic, Mistral, Ollama (local)	Gratuit, sans CB, latence très basse, modèle Llama 3.3 70B compétitif
Slack	Microsoft Teams, Discord, email	Webhook entrant trivial à configurer côté Slack vs OAuth complet sur Teams
Compte Gmail sandbox	compte de service GCP	Plus simple à expliquer en formation, et un compte de service ne peut pas avoir de boîte Gmail
n8n self-hosted	n8n Cloud, Make.com, Zapier	Coût zéro, portabilité (Docker), souveraineté des données

Lectures complémentaires recommandées

- **Documentation n8n** : <https://docs.n8n.io>
- **Documentation Groq** : <https://console.groq.com/docs>
- **Documentation OAuth Google** : <https://developers.google.com/identity/protocols/oauth2>
- **Slack Incoming Webhooks** : <https://api.slack.com/messaging/webhooks>

Estimer le coût en production

Si vous portez ce TP en production pour 1 utilisateur :

- **Infra** : ~ 5 €/mois (VPS basique) ou ~ 20 €/mois (n8n Cloud Starter)
- **LLM** : Groq gratuit jusqu'à plusieurs milliers de requêtes/jour ; OpenAI `gpt-4o-mini` ≈ 0,5-2 €/mois pour ce volume
- **Slack / Google** : 0 € (déjà payés via la suite Google ou l'abonnement Slack)

À comparer aux ~ 200 €/mois de Power Automate Premium pour un workflow équivalent (cf. TP 4.a). C'est le **delta économique** que nous discuterons en synthèse jour 3.

Et après ?

Une fois ces pré-requis validés, la séance pratique se déroulera en deux temps :

1. **Setup local n8n** (Docker Compose : n8n + Postgres + Qdrant) → cf. [tp4b-n8n-setup.md](#)
2. **Construction du workflow Morning Briefing** (déclaration des credentials + canvas) → cf. [tp4b-n8n-morning-briefing.md](#)

Bon courage pour la prep – **un setup propre = une séance fluide.**

– *Bassem Ben Hamed*